

SMART VEHICLE SAFETY

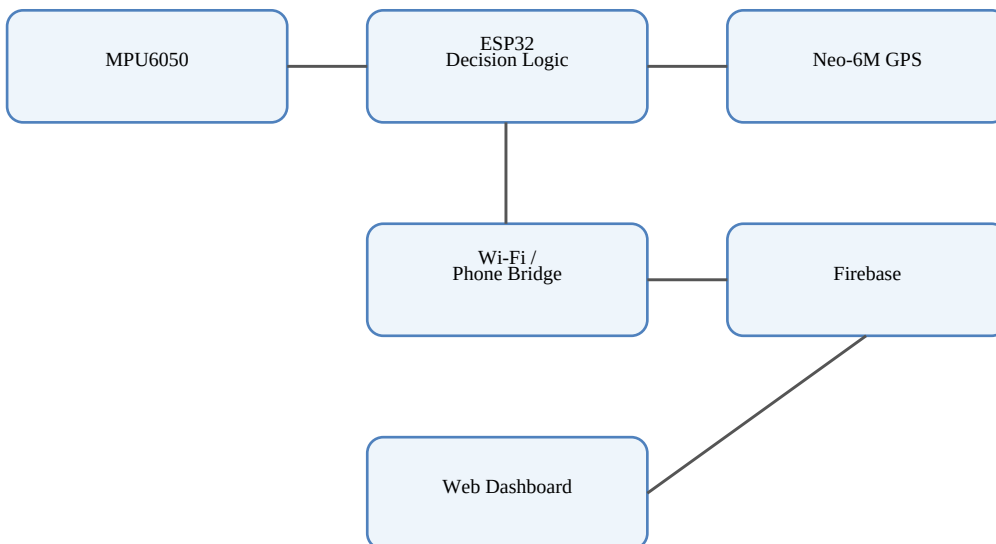
PHASE 3 — SYSTEM DESIGN REVIEW (SDR)

Smart Vehicle Accident Detection and Emergency Rescue System

Professional mentor-submission document

Scope: This document covers every Phase 3 item shown in the supplied review checklist: architecture, data flow, functional blocks, hardware/software partitioning, final hardware design, component selection, BOM, PCB considerations, software algorithms/protocols/database and simulation.

1. Overall Solution Architecture



Data flow: MPU6050 → ESP32 decision logic → accident state → Neo-6M GPS → Wi-Fi/phone bridge → Firebase → web dashboard/notification layer.

2. Data Flow Diagram

Stage	Input	Processing	Output
Sensing	Acceleration + gyro	Sample IMU	Motion data
Decision	IMU data	Impact + tilt + timing + cancellation	NORMAL/WARNING/ACCIDENT
Location	GPS NMEA	Parse fix/coordinates	Latitude/longitude
Cloud	Event + location	Database update	Realtime record
Dashboard	Firebase data	Render current state	Monitoring screen

3. Functional Block Diagram

Block	Function
MPU6050	Measures 3-axis acceleration and 3-axis angular velocity
ESP32	Reads sensors, executes detection logic and connectivity

Block	Function
Neo-6M	Provides geographic coordinates
Buzzer/LED	Immediate local warning
Switch	User cancellation / test input
Firebase	Stores and synchronizes event data
Web dashboard	Displays vehicle status, event and location

4. Hardware–Software Partitioning

Hardware	Software
MPU6050	Sampling, filtering, magnitude and tilt calculation
ESP32	State machine, timing and communication
GPS	UART/NMEA parsing
Buzzer/LED	Alert patterns
Switch	Debounce/cancellation logic
Battery/buck	Power delivery and monitoring

5. Final Block / Circuit Design

Module	ESP32 pin	Purpose
MPU6050 SDA	GPIO 21	I2C data
MPU6050 SCL	GPIO 22	I2C clock
Neo-6M TX	GPIO 16 (RX2)	GPS input
Neo-6M RX	GPIO 17 (TX2)	Optional GPS configuration
Buzzer	GPIO 25	Alert
LED	GPIO 26	Status
Switch	GPIO 27	Cancellation

6. Component Selection and BOM

Component	Reason	Status
ESP32	IoT controller with Wi-Fi/Bluetooth	Owned
MPU6050	Low-cost 6-axis IMU, I2C	Owned
Neo-6M GPS	Location data	Required/integration
Buzzer	Local warning	Owned
LED + resistor	Visual indication	Required if needed
Switch	Cancellation/test	Owned
Battery + buck	Portable power	Owned
PCB/perfboard	Physical integration	Phase 4

7. PCB Layout Requirements

Keep the IMU rigidly mounted and aligned with vehicle axes. Keep the ESP32 antenna area clear of metal/copper where practical. Keep I2C traces short, provide a common ground, protect the battery input, and provide test points for power, I2C and GPS serial lines.

8. Accident Detection Algorithm

State	Logic	Action
NORMAL	Monitor IMU continuously	No alarm
WARNING	Impact reaches warning level	Buzzer + LED; start cancellation timer
CANCELLED	User presses switch	Stop alert; log cancellation
ACCIDENT	Confirmed impact/tilt condition	Alarm + location/event publication

Prototype thresholds may use warning $\approx 3G$, confirmed $\approx 6G$ and tilt $\approx 80^\circ$ as starting engineering values. These are not universal accident standards and must be calibrated through controlled tests.

9. Communication Protocols

Interface	Protocol	Purpose
MPU6050 $\leftrightarrow$ ESP32	I2C	Sensor acquisition
GPS $\leftrightarrow$ ESP32	UART/NMEA	Location acquisition
ESP32 $\leftrightarrow$ Internet	Wi-Fi	Cloud connectivity
Web $\leftrightarrow$ Firebase	Firebase Web SDK / HTTPS	Realtime data

10. Database Structure

```
{
  "vehicles": {
    "DEVICE_ID": {
      "status": "NORMAL",
      "impactG": 0.82,
      "tiltDeg": 4.5,
      "gpsValid": true,
      "latitude": 13.0827,
      "longitude": 80.2707,
      "timestamp": 1760000000000
    }
  },
  "emergencyContacts": {
    "contact1": {"name": "Contact 1", "phone": "+91XXXXXXXXXX"},
    "contact2": {"name": "Contact 2", "phone": "+91XXXXXXXXXX"},
    "contact3": {"name": "Contact 3", "phone": "+91XXXXXXXXXX"}
  }
}
```

11. Flowchart / Software Sequence

Initialize $\rightarrow$ initialize MPU6050/GPS/GPIO $\rightarrow$ connect to network $\rightarrow$ read IMU $\rightarrow$ calculate impact/tilt $\rightarrow$ normal? continue $\rightarrow$ warning? alert and wait for cancellation $\rightarrow$ cancelled? return to monitoring $\rightarrow$ otherwise confirm accident $\rightarrow$ read GPS $\rightarrow$ publish event $\rightarrow$ retry/handle connectivity $\rightarrow$ continue monitoring.

12. Simulation and Initial Results

Use Wokwi for ESP32 + MPU6050 simulation. Where the selected simulator does not provide the required GPS component, use mock/fixed GPS data and explicitly document that GPS hardware is validated separately. Submit screenshots of normal readings, warning, cancellation, confirmed event and dashboard/cloud update.

13. Phase 3 Review Checklist

Deliverable	Status
Overall Solution Architecture	■
Data Flow Diagram	■
Functional Block Diagram	■
Hardware–Software Partitioning	■
Final Block Diagram	■
Circuit Schematic	■
Component Selection	■
BOM	■

Deliverable	Status
PCB Layout if applicable	■
Flowcharts	■
Algorithms	■
Communication Protocols	■
Database Structure	■
Simulation	■
Initial Results	■

Professional note: current system positioning is **rule-based multi-sensor accident detection**. Do not claim AI/ML unless a trained and validated model is actually implemented.